MoveInSync Backend Case Study 2026

SDE – Java Backend Hiring Assessment · Advanced Track (IIIT & NIT) · Case Study 2

Case Study 2: Hierarchical Tenant Configuration with Live Propagation

MoveInSync runs hundreds of corporate clients on one platform, and each client tunes hundreds of settings: booking cut-off time, maximum ride time, night-safety window, escort rules, cab types allowed. A setting can be defined at several levels, GLOBAL then CLIENT then OFFICE then SHIFT, and the most specific value wins. Forty services read these settings on almost every request, so they cannot call a config server over HTTP each time. Your job is to build the configuration service and the client library that every other service embeds, so reads are instant and changes still reach everyone within seconds.

I. Model Settings & Levels

- Definitions — each setting has a key, a type (number, duration, time window, enum, list), validation rules and a global default.
- Overrides — values can be set at CLIENT, OFFICE or SHIFT level. Resolution is most-specific-wins, and "explain" mode shows which level supplied the value.
- Versioned Changes — every change records author, reason and a monotonically increasing version number.

II. The Read Path

- Local Snapshot — the client library keeps an in-memory snapshot and answers reads in microseconds with no network call.
- Bounded Staleness — a published change is visible in every connected client within 5 seconds. Use push (WebSocket, server-sent events or long-polling), plus a periodic version check that heals any missed push.
- Reconnect Storm — after a server restart, 500 clients reconnecting at once must not overload it. Use jitter and send only what changed since each client's last version.

III. Safe Changes

- Atomic Change Sets — a change to the night window's start and end is applied together or not at all. No client may ever see the new start with the old end.
- Staged Rollout — a change can go to two chosen offices first and be promoted to everyone later.
- Instant Rollback — roll back to any previous version in one call, with a diff of what changes.

IV. Failure Behaviour

- Server Down — clients keep serving their last good snapshot and can start up from a copy saved on disk.
- Bad Value — an invalid value (a cut-off of −30 minutes, a window that ends before it starts) is rejected before it is published.

Example Use Case

• An operator changes the booking cut-off for one Bengaluru office from 60 to 30 minutes. Within 5 seconds the booking service starts accepting bookings 45 minutes before departure for that office only. Every other office still uses 60.

• The config server is killed. Every service keeps answering reads from its snapshot, and the change made just before the crash is still applied everywhere once the server returns.

Key Points to Keep in Mind

- This is a cache-invalidation problem. Write down your consistency model in one paragraph.
- Versions must only move forward. Explain what a client does when it receives version 41 after version 43.
- Atomic change sets are the subtle case. Show a test that reads continuously during a change and never sees a mixed state.
- **Prove it:** benchmark local reads, measure the propagation time to 200 simulated clients, and include a kill-the-server test.

Plus Points in Implementation (Overall Evaluation Criteria)

1. **Authentication:** Implement robust authentication and role-based access (employee vs admin vs operator), with tenant isolation so one client can never read another client's data.
2. **Cost Estimation – Time and Space:** Analyse the time and space complexity of your core logic, and back it with a small benchmark. State where the system stops scaling and why.
3. **Handling System Failure Cases:** Implement fault-tolerant mechanisms for crashes, timeouts, duplicate and out-of-order messages, and partial failures. Show at least one failure-injection test.
4. **Object-Oriented Programming (OOPS):** Choose a robust OOPS language (Java preferred). Apply encapsulation, inheritance and polymorphism for maintainable, extensible code with clean controller / service / repository layering.
5. **Trade-offs in the System:** Clearly define and document the trade-offs you made, especially consistency vs availability vs latency. Explain what you deliberately left out.
6. **System Monitoring:** Add metrics and structured logging. Expose the counters an on-call engineer would need at 3 AM (lag, error rate, rejections, retries, queue depth).
7. **Caching:** Integrate caching for hot data with a sensible eviction policy. Say exactly what can become stale, for how long, and how staleness is bounded.
8. **Error and Exception Handling:** Build a robust error-handling framework with meaningful, non-leaking error messages. No personal data (phone, email, home location) in logs or error responses.

What "Advanced Track" Means

- **Proven guarantees** – every hard rule in your case study has an automated test, including at least one concurrency test and one failure test.
- **Measured performance** – a small load test or benchmark with throughput and p50 / p99 latency, reported in the README with the machine you ran it on.
- **Design note** – one or two pages with an architecture diagram, one sequence diagram for the hardest flow, and a failure-mode table (what fails, what the user sees, how the system recovers).
- **One-command run** – the project starts with a single command (for example docker compose up), including any database or message broker it needs.

Instructions

1. **Read and Understand the Problem Statement:** Carefully read the case study. Understand the requirements, inputs, expected outputs, the hard rules and the scale targets.
2. **Choose a Programming Language:** Java with Spring Boot is preferred. C#, Go, Kotlin or Python are acceptable if you can defend the choice.
3. **Design Your Solution:** Plan first. Decide your data model, the main components, the APIs and the failure modes before writing code. Put this in the design note.
4. **Write the Code:** Write clean, layered code with meaningful names. Keep the core algorithm in plain classes that can be tested without a web server or database.
5. **Test Your Code:** Test normal inputs AND edge cases. Add unit tests for the core logic, plus the concurrency, failure and benchmark tests described in your case study.
6. **Document Your Code:** Add a README with how to run it, your assumptions, the benchmark results, and a short note on the trade-offs you made.
7. **Submit Your Solution:** Push to a public GitHub repository and share the link. Keep a meaningful commit history, not one giant commit.
8. **Demonstration:** Add a short demo video showing the main flows, one failure scenario being handled live, and a few sample API calls (Postman / curl).